Drava: An Event-Driven Runtime for Scientific Pipelines

Ahmedur Rahman **Shovon**
Postdoctoral Appointee

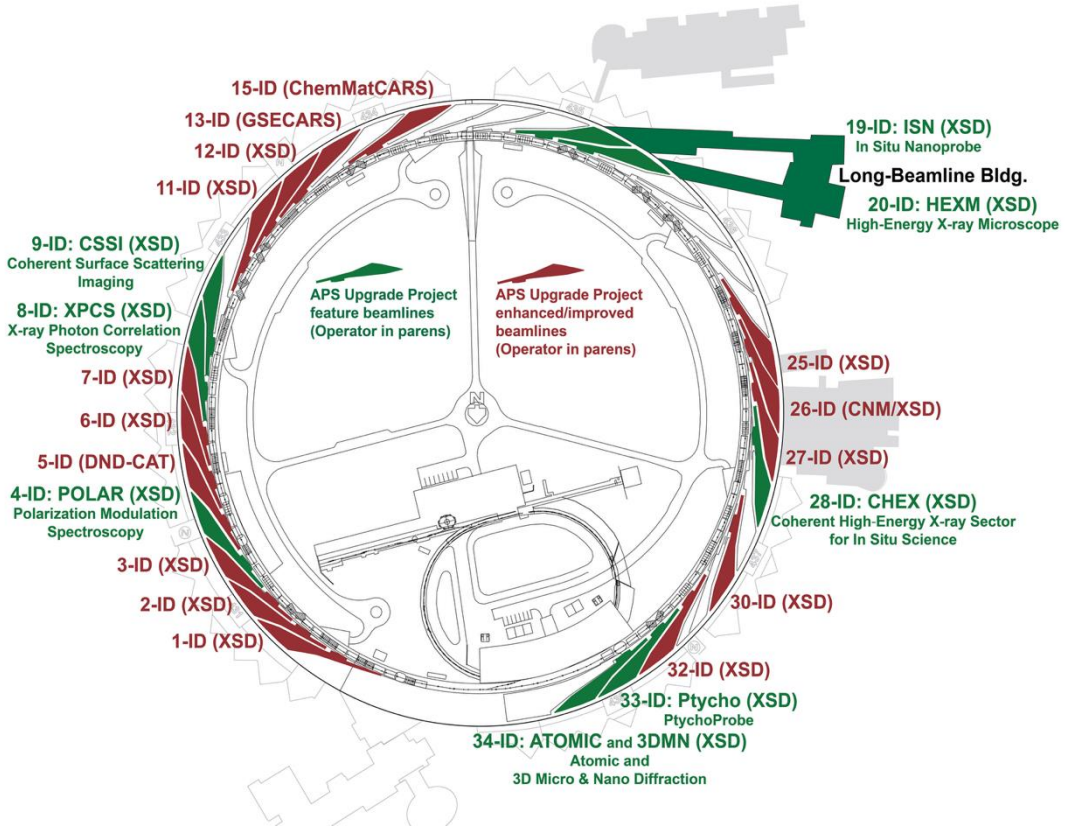
Mathematics and Computer Science Division
Argonne National Laboratory

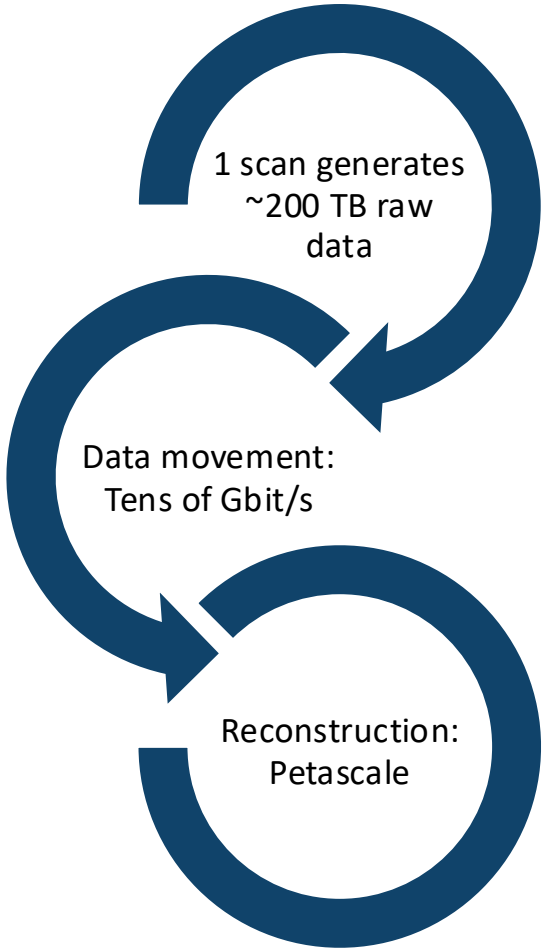


U.S. DEPARTMENT of ENERGY | Office of Science

Energy.gov/science

Scale of Modern Scientific Workflows (XRS)





Future detectors >= 1000X data rates in 2 decades

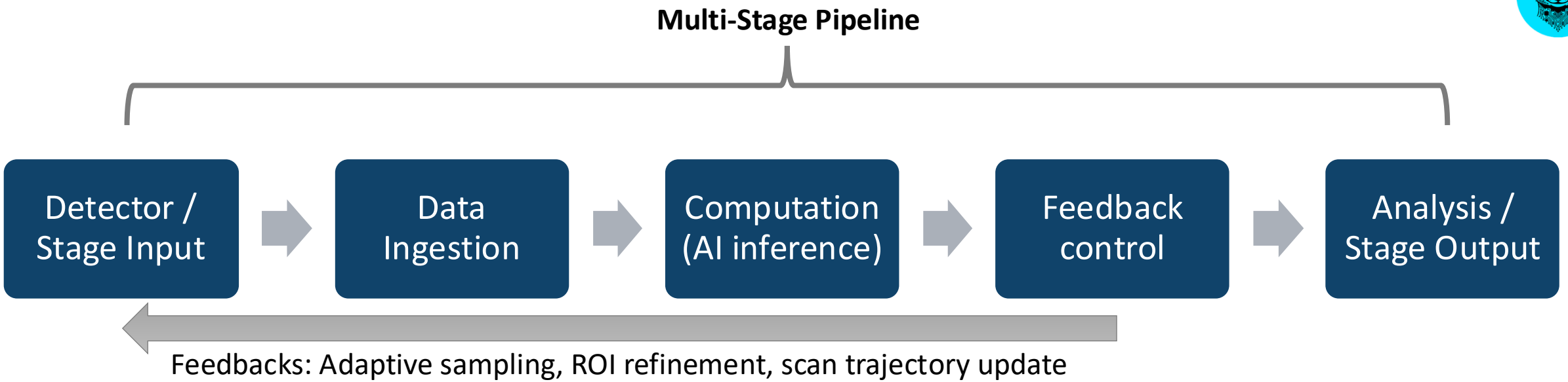
Data reduction and transport

Analysis and feedback control

Image source: https://www.aps.anl.gov/APS-Upgrade/Feature-Beamlines

1. Advanced Photon Source Strategic Plan 2025-2029

Workflow Characteristics: Streaming, Multi-Stage, Feedback-Loop



Application Knobs: Batch size, model params, Sampling / ROI, Stage configs

Runtime Knobs: Concurrency, Transport choice, Placement

Tightly coupled: computation ↔ data transfer

Combinatorial configuration space (hyperparameters)

1. Buurlage et al. Real-time reconstruction and visualisation towards dynamic feedback control during time-resolved tomography experiments at TOMCAT. Scientific reports, 9(1), 18379.
2. Yongho et al. Charon: An end-to-end infrastructure for connecting ai@edge to hpc. IPDPSW 2025
3. Babu et al. Deep learning at the edge enables real-time streaming ptychographic imaging. Nature Communications, 14(1):7059, 2023

Custom HWs Solves Latency, Not Flexibility



Hardware Centric

- FPGA-based DAQ
- ASIC-based trigger logic (XPOL)
- Detector specific chain (EIGER)

✓ Low latency
✗ Hard to reconfigure
✗ No composability across experiments
✗ No end-to-end observability

Software Managed

- Reusable function blocks
- General-purpose resources
- Reconfigurable as methods evolve

✓ Flexible & composable
✓ Observable end-to-end
✓ Adaptable to new detectors
⚠ Should be generic

1. Windisch et al. FPGA-based real-time data acquisition for ultrafast x-ray computed tomography. IEEE Transactions on Nuclear Science, 2021.
2. Vernon et al. Development of a high-rate frontend asic for x-ray spectroscopy and diffraction applications. IEEE Transactions on Nuclear Science, 2020.

Drawbacks of Current Software Pipelines



Lack of Runtime Abstraction

- No unified model for composing multi-stage pipelines
- Adhoc, tightly coupled implementations

Weak Instrumentation

- Limited stage level and pipeline level runtime metrics
- No transport aware orchestration

Config Explosion

- Lack of systematic approach to configuration tuning
- No explicit control over application and runtime level knobs

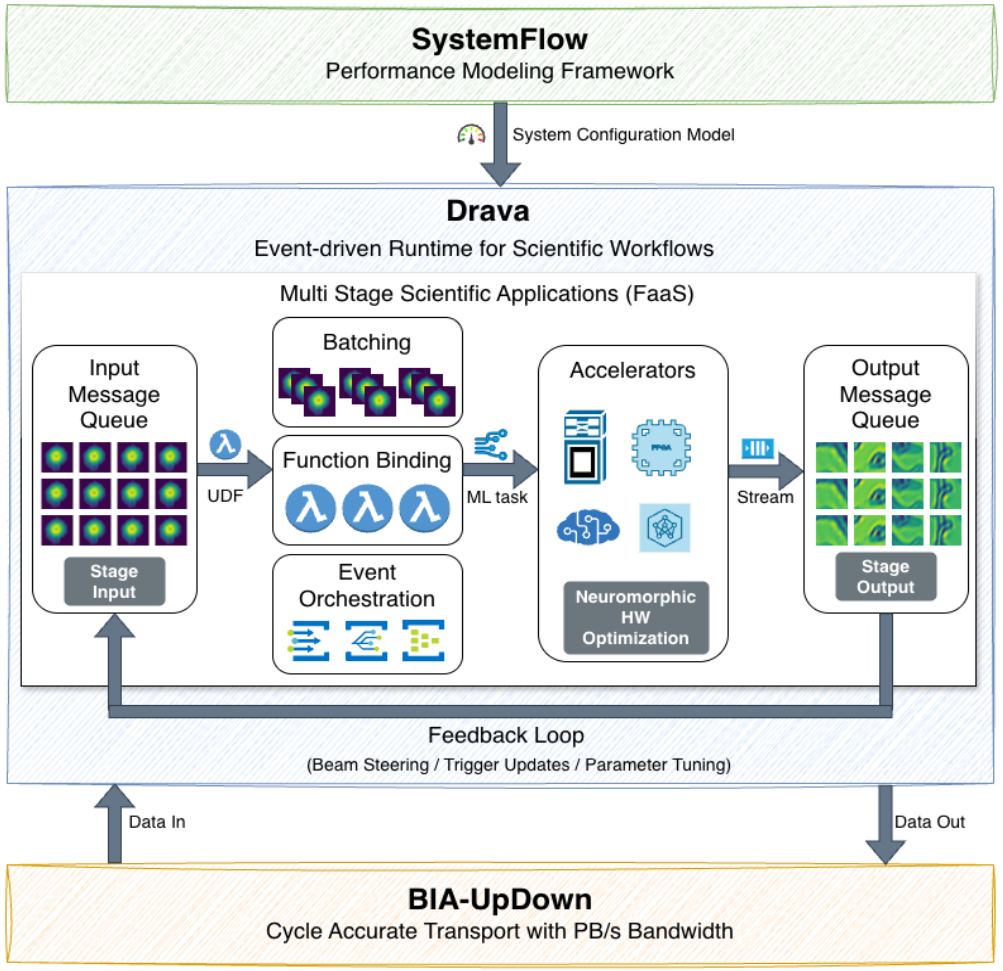
1. Numaflow https://github.com/numaproj/numaflow
2. Veseli, S., Hammonds, J., Henke, S., Parraga, H., Frosik, B., & Schwarz, N. (2025). PvaPy streaming framework for real-time data processing. Journal of synchrotron radiation, 32(Pt 3), 823–836.

Introduction



*https://commons.wikimedia.org/wiki/File:The_Drava_river_and_Maribor.JPG

Drava: An Event-Driven Runtime for Scientific Pipelines



Drava serves as the runtime orchestration layer. SystemFlow provides system-level performance modeling and configuration models that guide execution, while Drava runs scientific workflows using an event-driven function-as-a-service model. Data movement across pipeline stages is handled by the cycle-accurate BIA-UpDown transport layer.

- **Scientific Achievement**
  - Establishes a Function-as-a-Service infrastructure to model and evaluate integrated hardware-software event-driven scientific instruments/pipelines.
- **Significance and Impact**
  - Enables integration of system-level modeling and novel transport and accelerator simulations into end-to-end pipelines for high-throughput validation studies of HEP/XRS applications.
  - Provides a software orchestration layer for integrating adaptive, feedback control optimizations, and steering policies of end-to-end pipelines.
- **Research Details**
  - Implements a message-driven function-as-a-service infrastructure that supports batching and event orchestration, integrating cycle-accurate simulation of Updown transport and AI4Science accelerator models.
  - Provides a flexible runtime abstraction layer and programming model for expressing HEP/XRS application pipelines as high-throughput dataflow pipelines.

Drava's Contributions



Event-driven abstraction for streaming workflows



Execution orchestration across heterogeneous resources

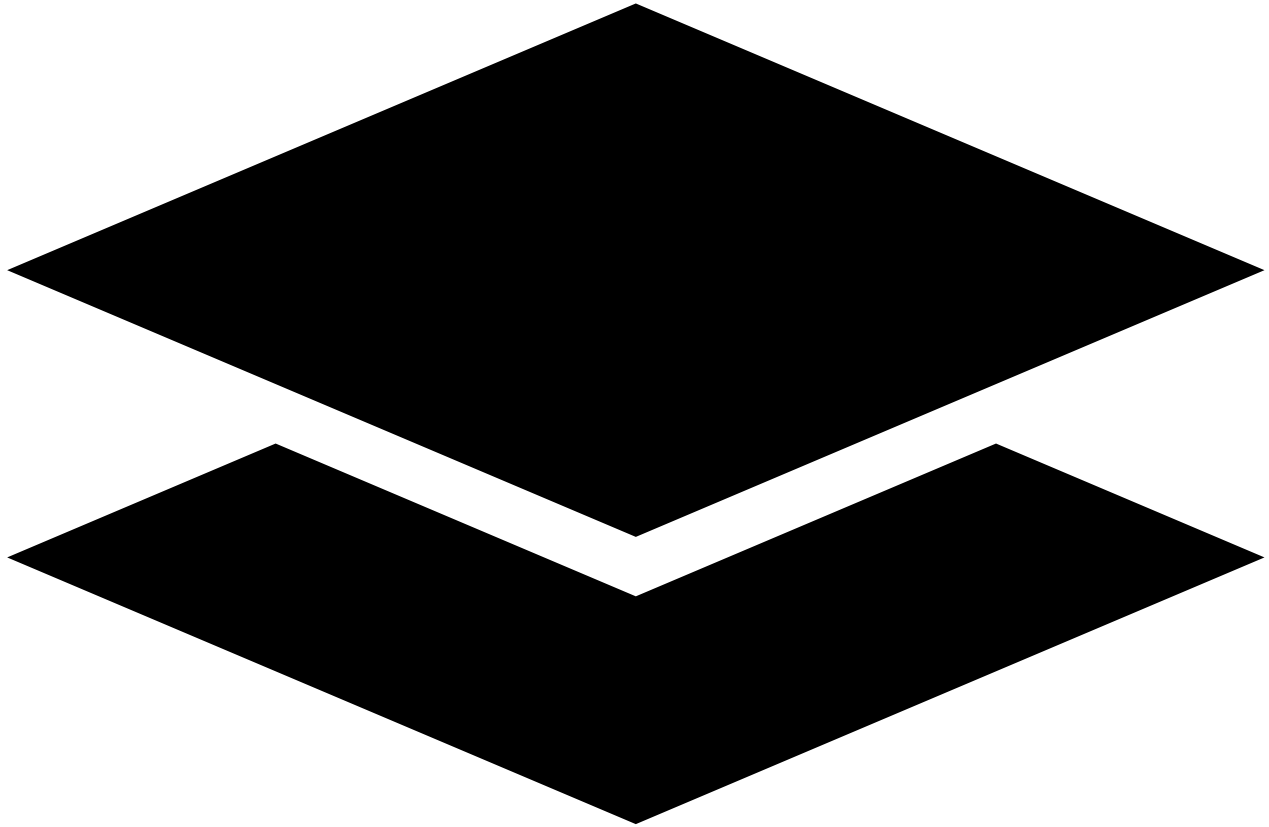


End-to-end observability for latency and energy



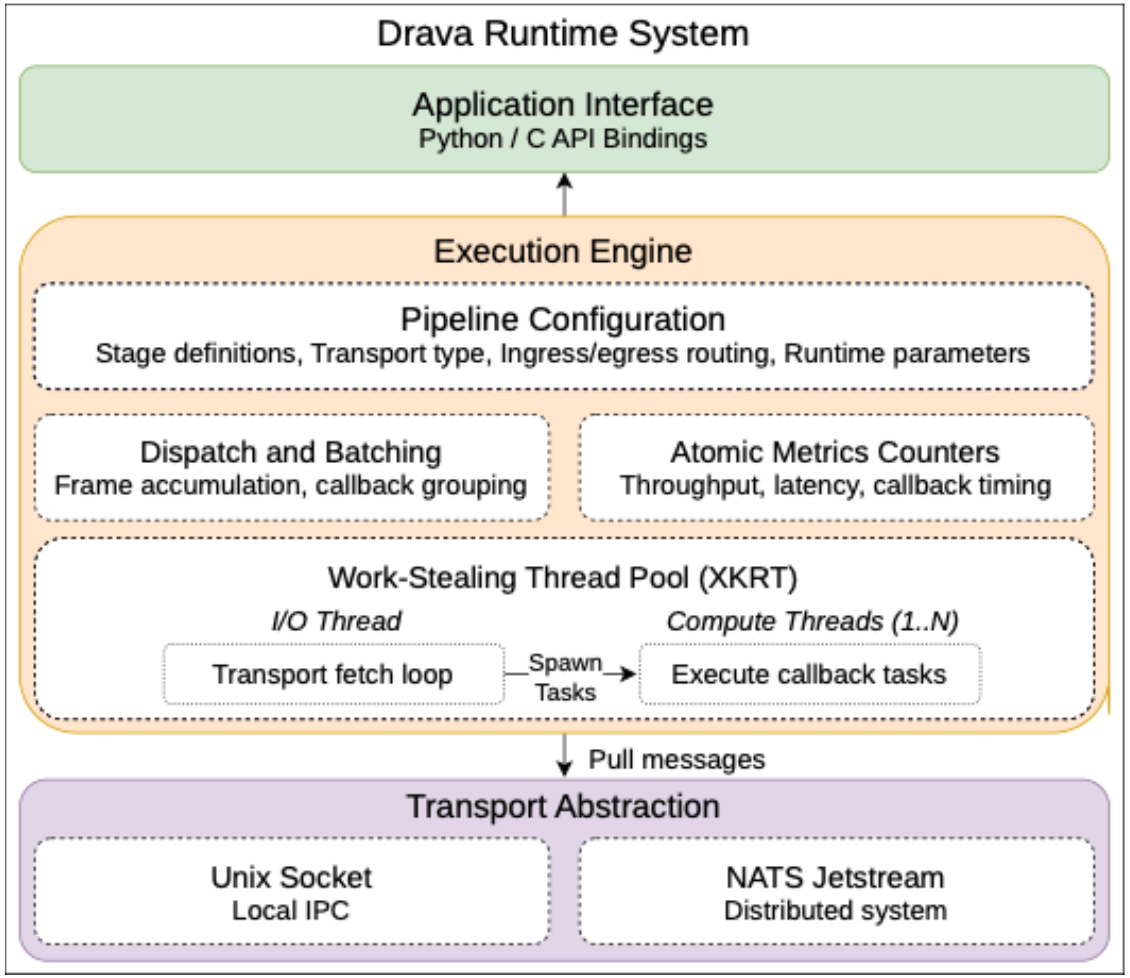
Configuration optimization via guided exploration

System Design
&
Runtime Architecture



System Architecture

- Layered abstraction model
- Event-driven execution engine
- Pub-Sub programming model
- Built-in dispatch and batching
- End-to-end performance metrics
- Pluggable transport layer



Scientific Application Workflow using Drava

1 Interface
Drava Python API

examples > tomogan > demo.py > ...

```python
import drava

# User-defined stage/callback
def func(frames):
    for frame in frames:
        ...

# Initialize Drava runtime
drava.init()

# Register callback as a pipeline stage
drava.register_routine_py(func)

# Start event loop
drava.listen_py()

# Shutdown runtime and release resources
drava.deinit()
```

User Application (app.py)

2 Config
YAML Parser

examples > tomogan > ! pipeline.yaml

```yaml
pipeline:
  name: tomogan_single_stage

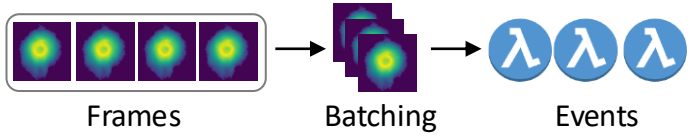
transport:
  type: nats
  nats_url: nats://127.0.0.1:4222

publisher:
  rate_hz: 2000

stages:
  - name: stage1
    runtime:
      threads: 4
      callback_batch: 16
    ingress:
      stream: FRAMES
      subject: frames.raw
      durable: drava_tomogan_stage1
      fetch_batch: 16
      fetch_timeout_ms: 200
```

Pipeline & stage configuration

3 Execute
Drava Runtime



Work-Stealing Scheduler (XKRT)

Worker 0 GPU 0 ↔ Worker 1 GPU 1 ↔ Worker 2 GPU 2

Multi-GPU Execution

GPU 0 | GPU 1 | GPU 2 | GPU 3

Resource scheduling & work distribution

4 Observability
Metrics & Reporting

Collected Metrics

- End-to-end latency
- Throughput (events/s)
- GPU utilization (%)
- Transport overhead (s)

Report

Export metrics in JSON / CSV / memory

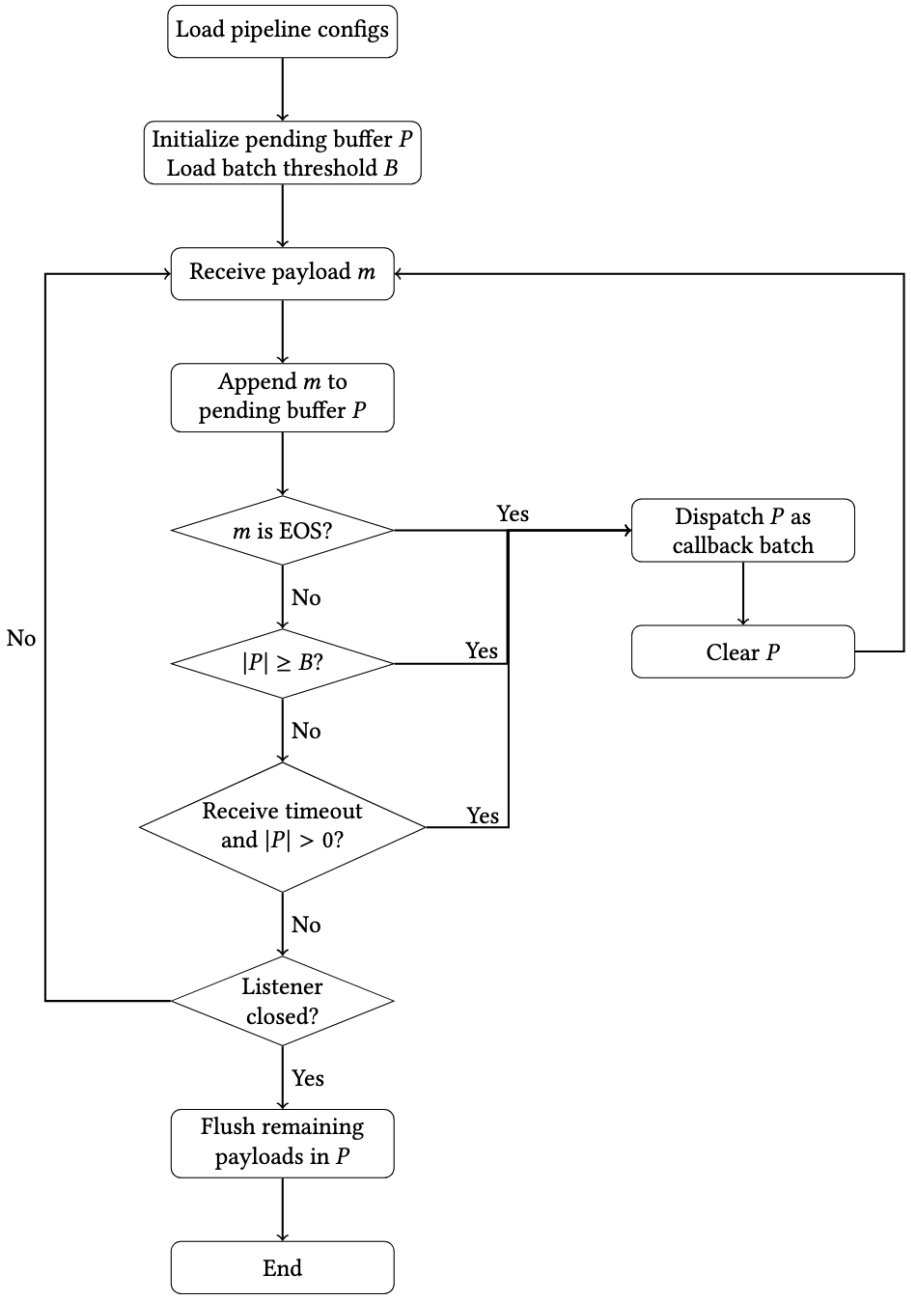
Feedback for monitoring & adaptive optimization

1. Wire Drava API in application code
2. Provide configuration in YAML
3. Drava executes tasks using XKRT across GPUS
4. Collect & report runtime metrics

Closed-loop for optimization

Batching in Streaming Pipeline

- Event-driven ingestion accumulates payloads into a receiving buffer
- Dispatch triggered by batch size B, timeout, or EOS
- Larger batches → higher throughput
  Smaller batches → lower latency



Per-stage metrics reported by Drava

Metric	Meaning	Source
`rx_msgs`	Callback batches processed	Counter
`rx_items`	Non-EOS frames received	Counter
`rx_bytes`	Input payload bytes	Counter
`tx_msgs`	Published payloads	Publish
`tx_bytes`	Published bytes	Publish
`cb_avg_ms`	Mean callback-batch duration	Timer
`publish_total_s`	Cumulative publish time	Timer
`stage_avg_ms`	Mean dispatch-to-publish time	Samples
`stage_total_fps`	Frames per stage interval	Snapshot

Evaluation

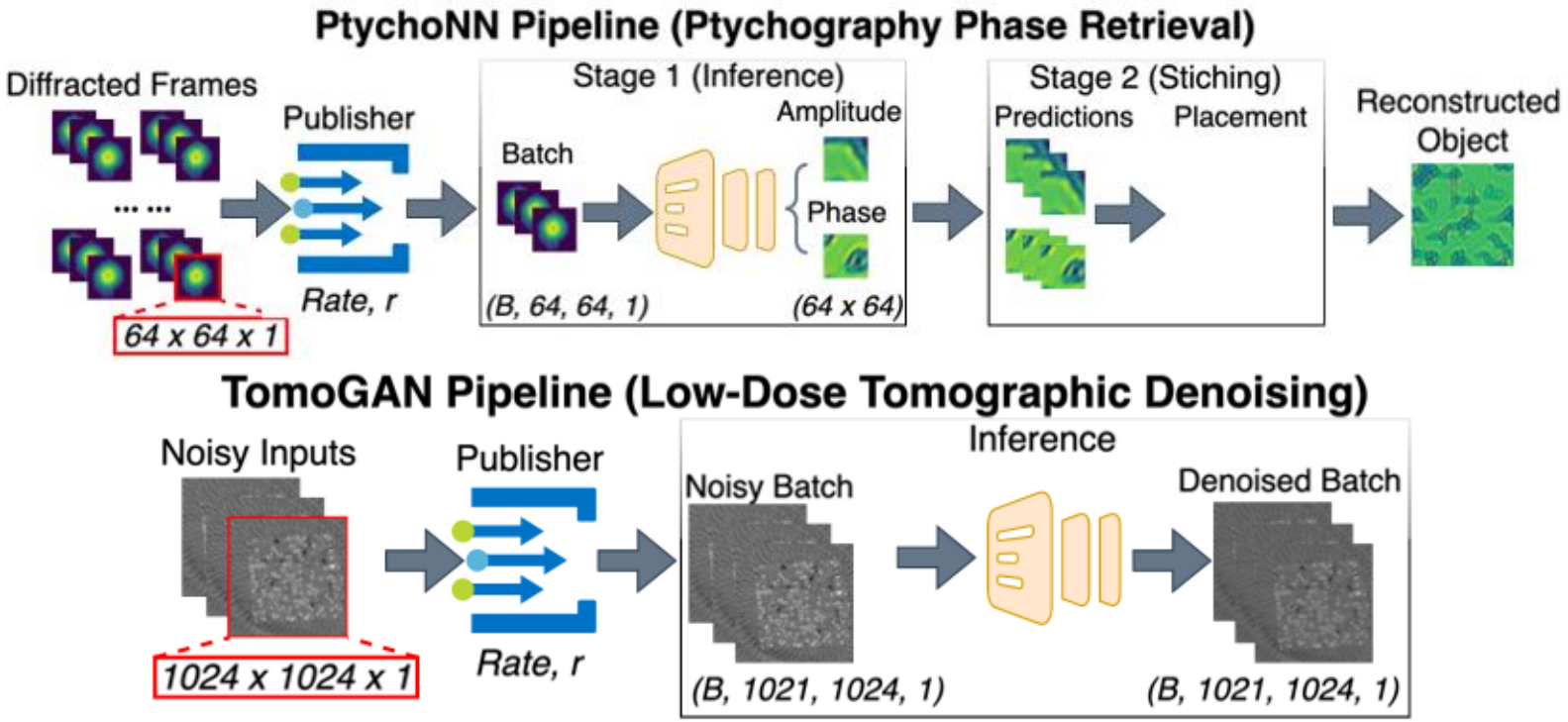


Evaluation Environment

Category	Configuration
System	JLSE, Argonne National Laboratory [27]
CPU	Dual-socket AMD EPYC 7532, 32 cores/64 threads
Memory	256 GB DDR4
GPU	NVIDIA A100 40 GB PCIe
Compiler	LLVM/Clang 23.0.0git, C++20
GPU stack	CUDA 12.3, cuDNN 9.19
Python stack	Free-threaded CPython 3.13, NumPy 2.3
ML stack	TensorFlow 2.20, Keras 3.12
Runtime deps.	XKRT, `yaml-cpp`, NATS C, SWIG 4.4.1
Transport (NATS)	NATS Server v2.11 with JetStream

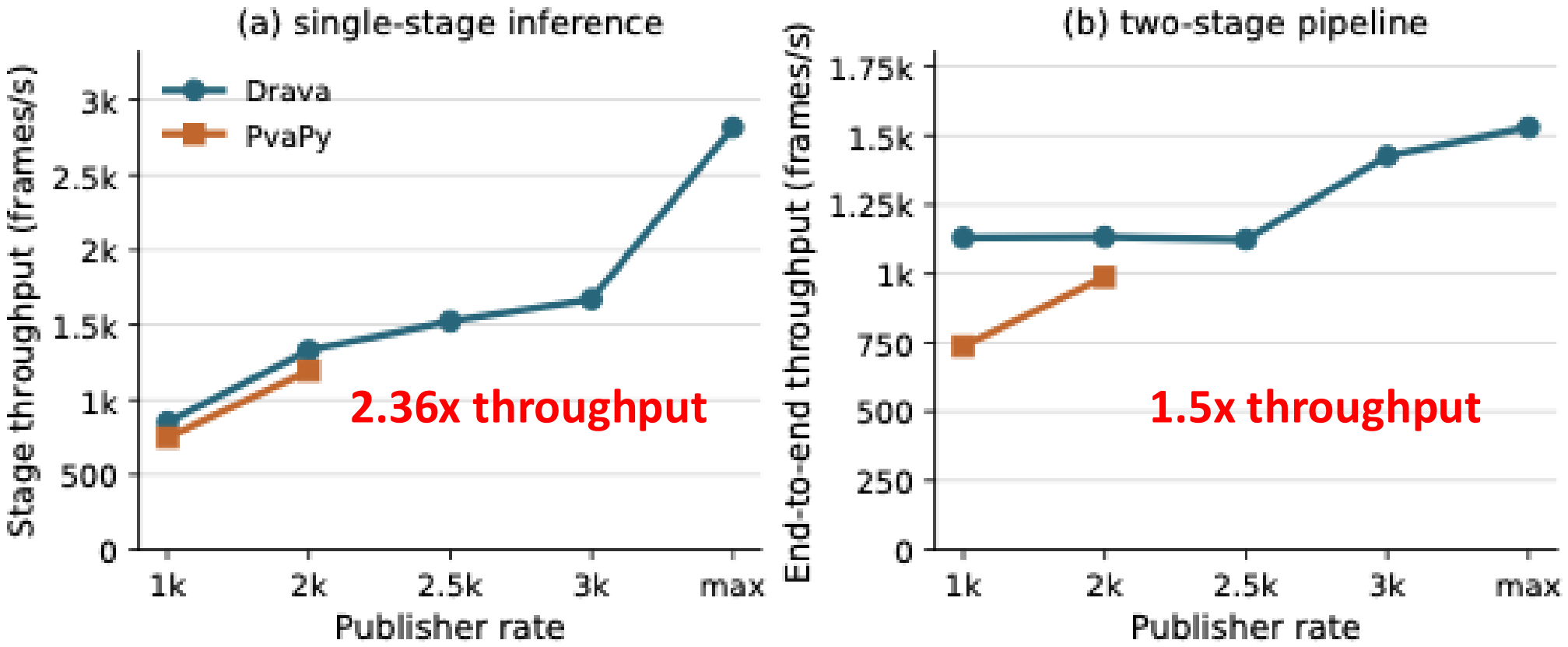
Evaluation Environment

- Applications: PtychoNN (2-stages), TomoGAN (single-stage)
- Dataset and pretrained models: HuggingFace



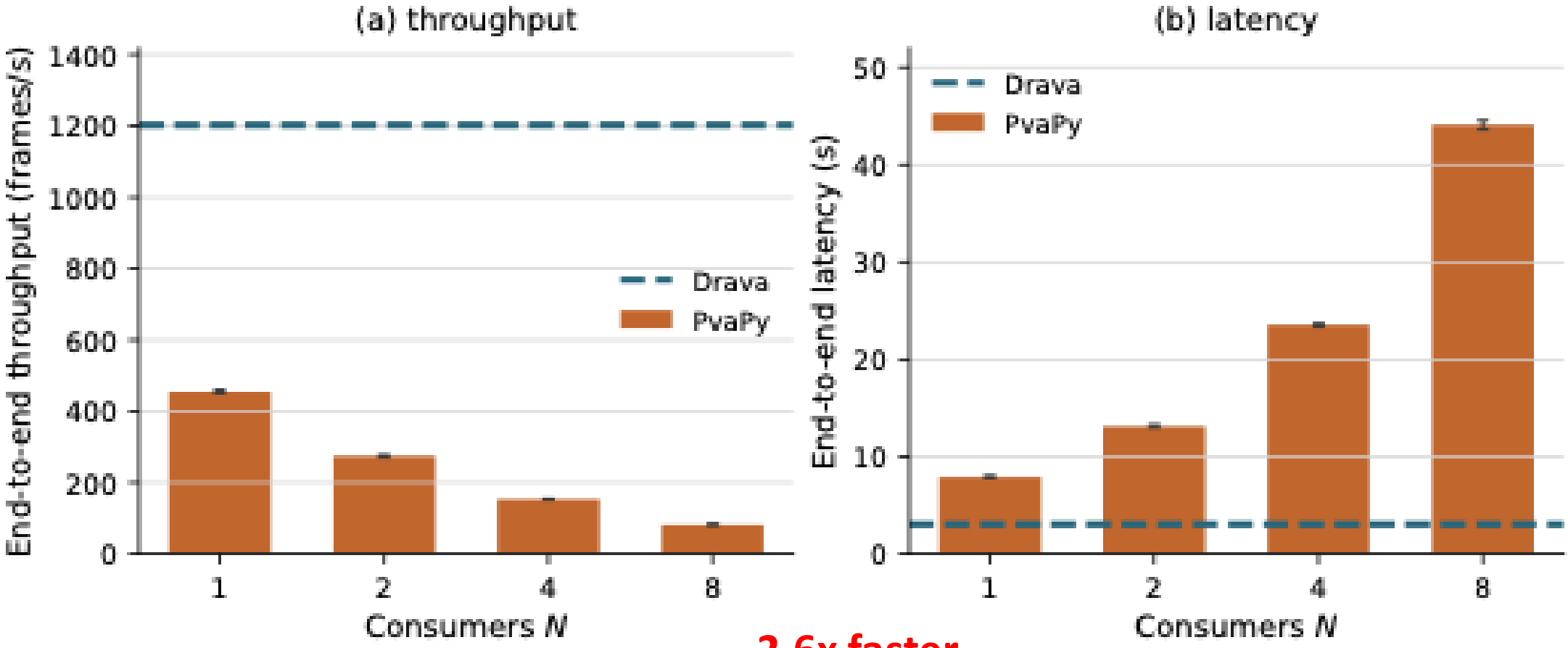
1. Cherukara, M. J., Zhou, T., Nashed, Y., Enfedaque, P., Hexemer, A., Harder, R. J., & Holt, M. V. (2020). AI-enabled high-resolution scanning coherent diffraction imaging. Applied Physics Letters, 117(4).Vernon et al. Development of a high-rate frontend asic for x-ray spectroscopy and diffraction applications. IEEE Transactions on Nuclear Science, 2020.
2. Liu, Z., Bicer, T., Kettimuthu, R., Gursoy, D., De Carlo, F., & Foster, I. (2020). TomoGAN: low-dose synchrotron x-ray tomography with generative adversarial networks: discussion. Journal of the Optical Society of America A, 37(3), 422-434.

Compare with SOTA (PvaPy)



1. Veseli, S., Hammonds, J., Henke, S., Parraga, H., Frosik, B., & Schwarz, N. (2025). PvaPy streaming framework for real-time data processing. *Journal of synchrotron radiation*, *32*(Pt 3), 823–836.

Compare with SOTA (PvaPy)

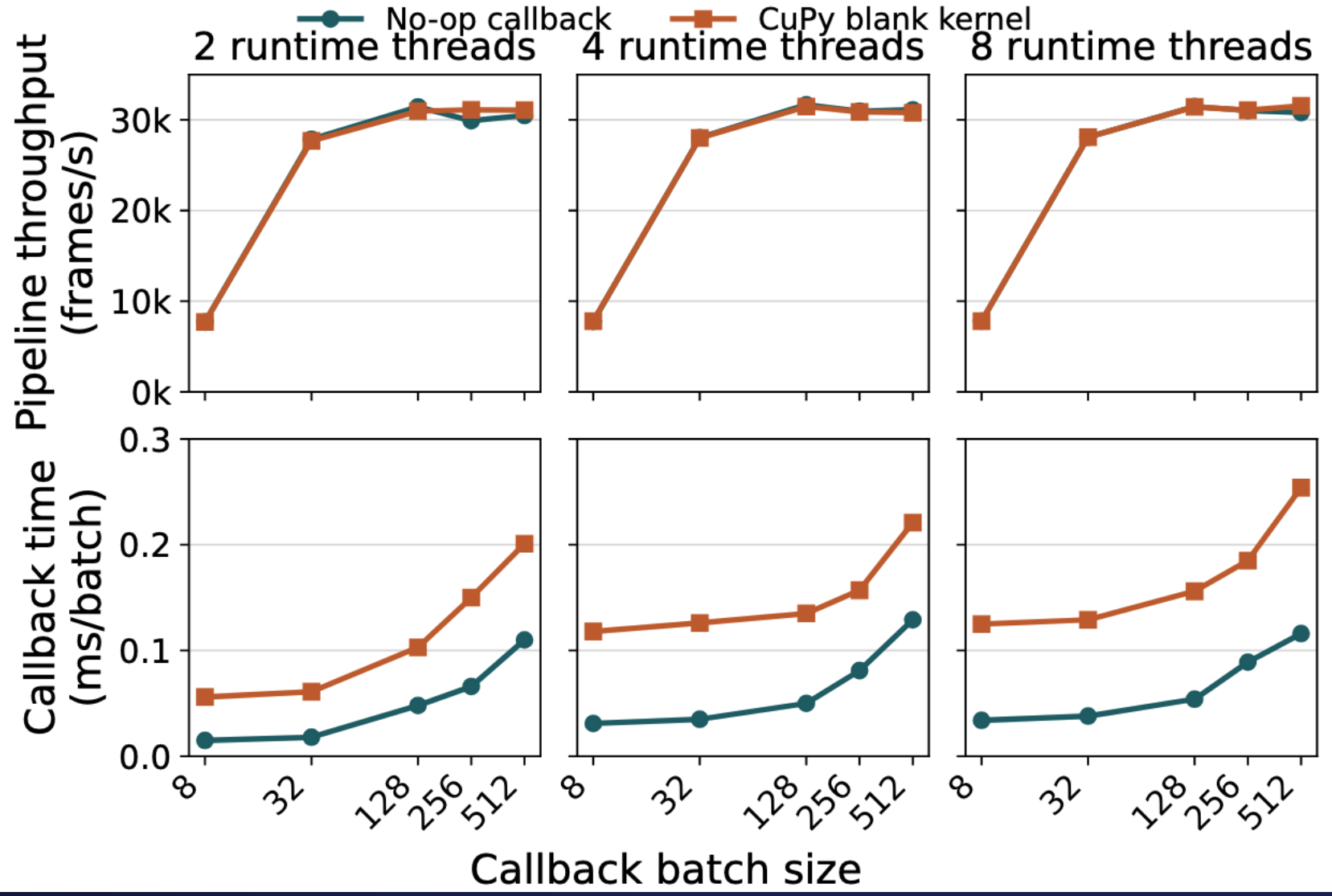


2.6x faster

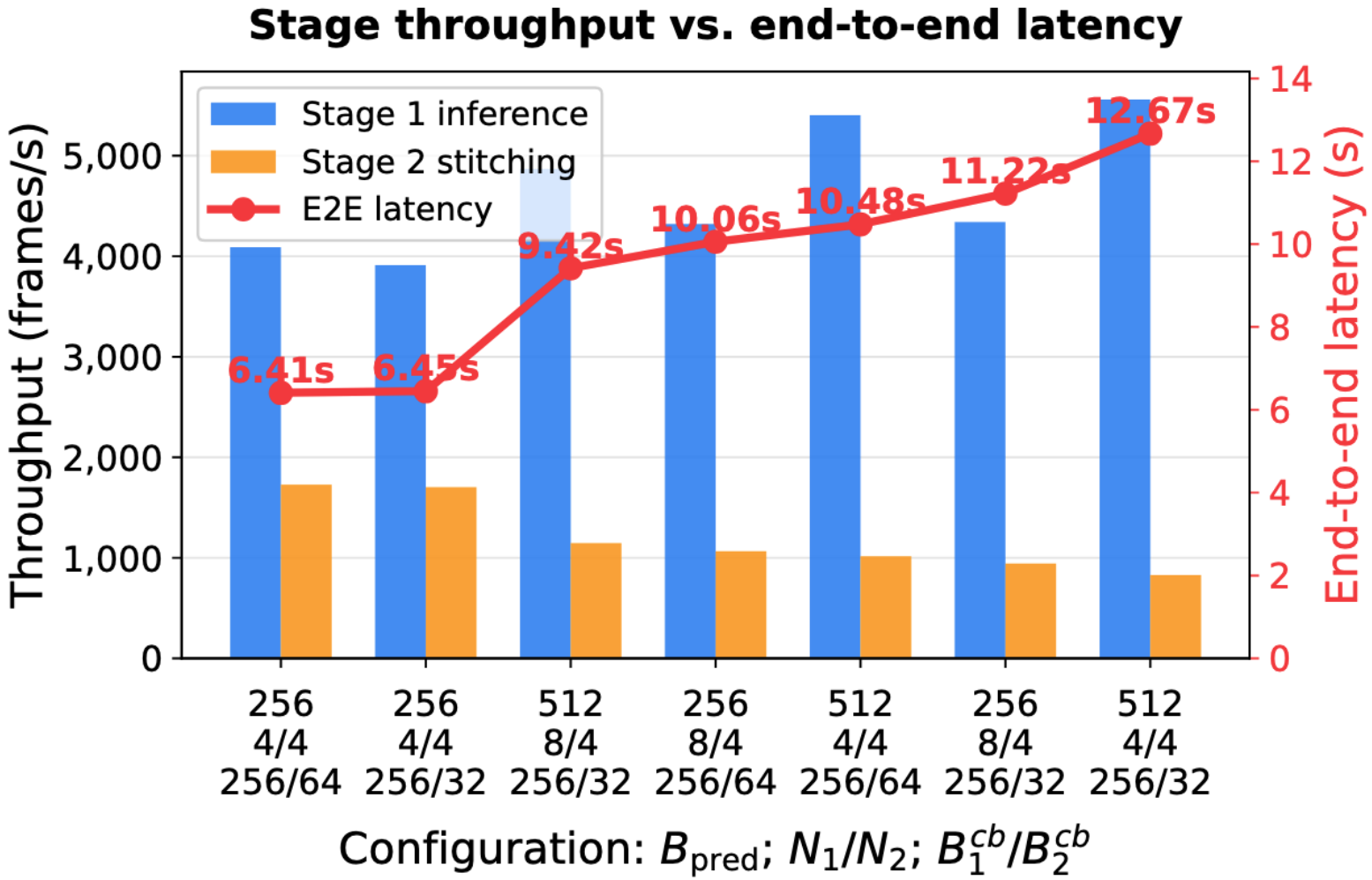
1. Veseli, S., Hammonds, J., Henke, S., Parraga, H., Frosik, B., & Schwarz, N. (2025). PvaPy streaming framework for real-time data processing. *Journal of synchrotron radiation*, *32*(Pt 3), 823–836.

Raw Performance

31k frames/s throughput

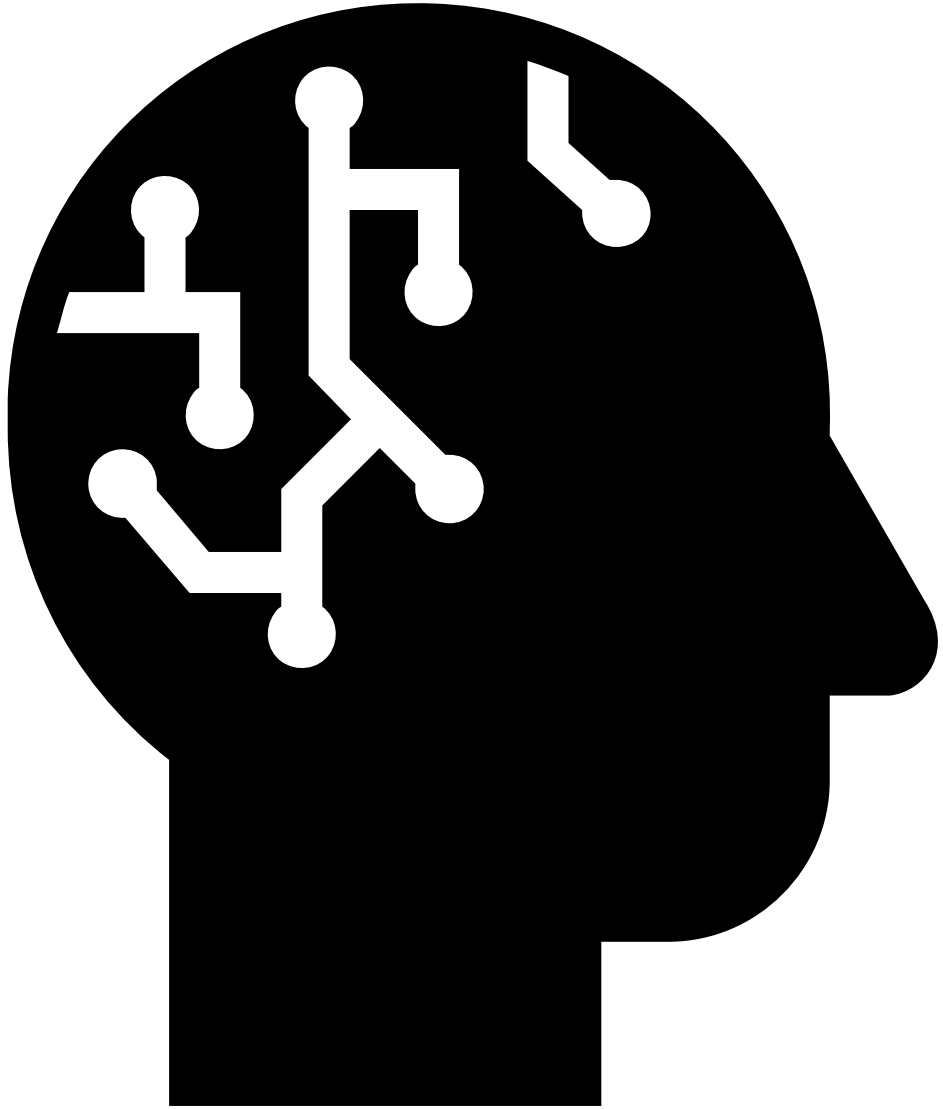


Configuration Determines Pipeline Bottleneck

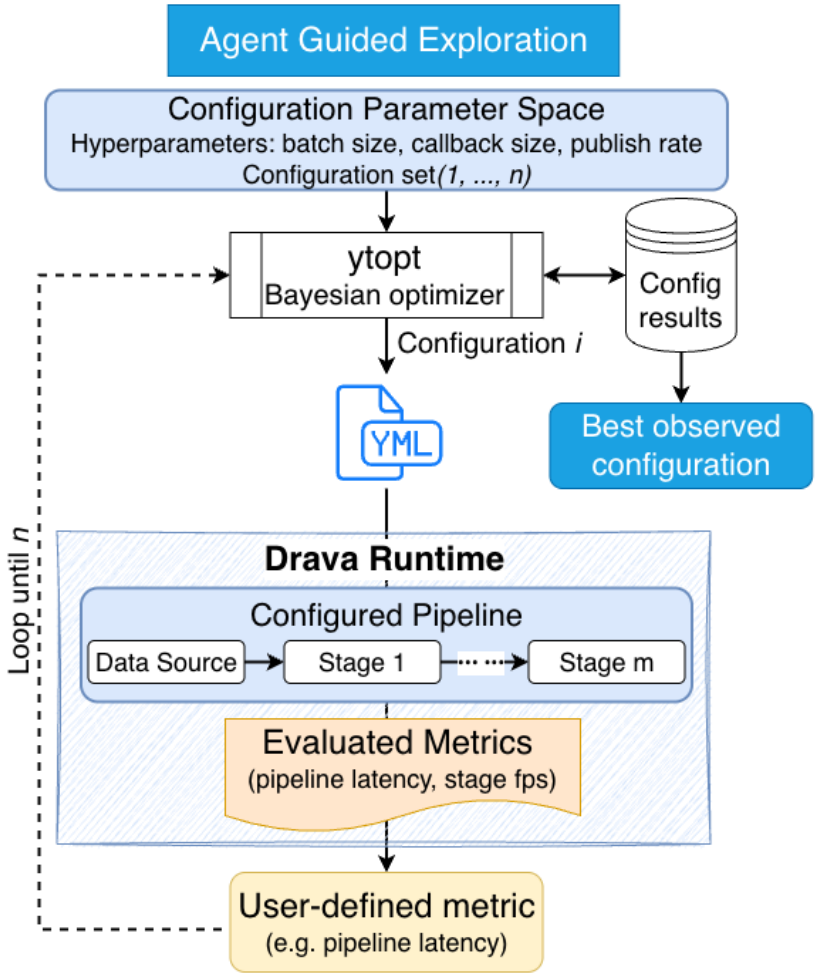


PtychoNN two stage analysis: Per-stage throughput and end-to-end latency across pipeline configurations. Bars show Stage 1 (inference) and Stage 2 (stitching) throughput (left axis); the line shows pipeline latency.

Agentic Exploration

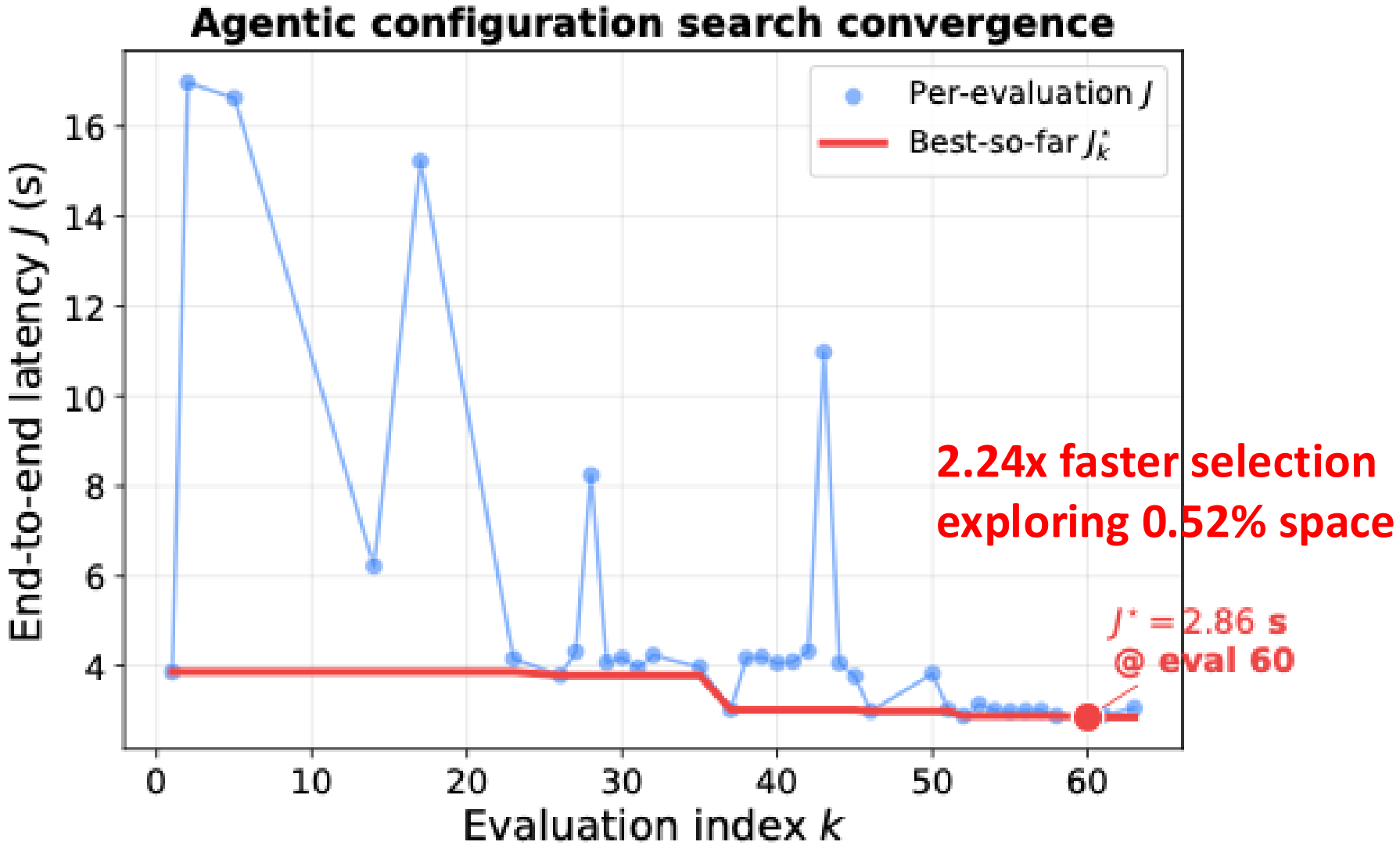


Agentic Exploration of Hyperparameters

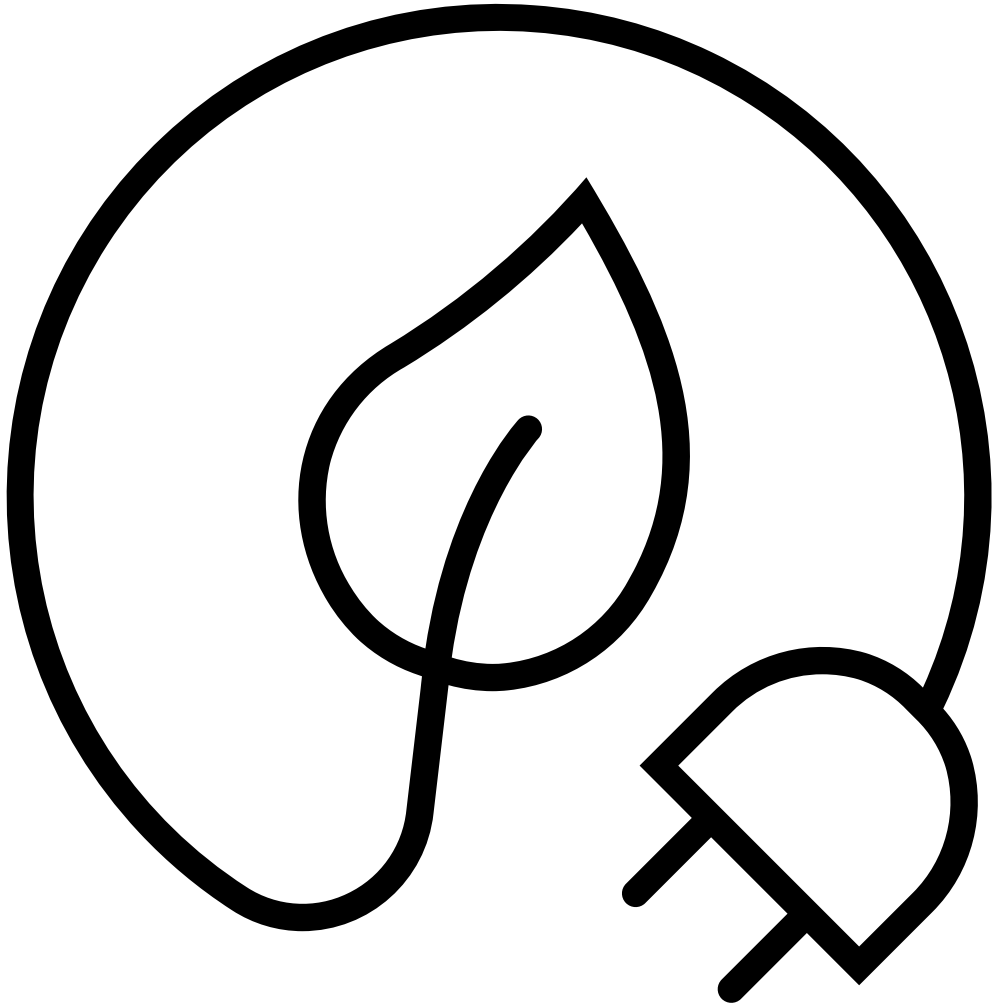


1. X. Wu, P. Balaprakash, M. Kruse, J. Koo, B. Videau, P. Hovland, V. Taylor, B. Geltz, S. Jana, and M. Hall, "ytopt: Autotuning Scientific Applications for Energy Efficiency at Large Scales", Concurrency and Computation: Practice and Experience, Vol. 37 (1): e8322, Jan. 2025
2. https://github.com/ytopt-team/ytopt

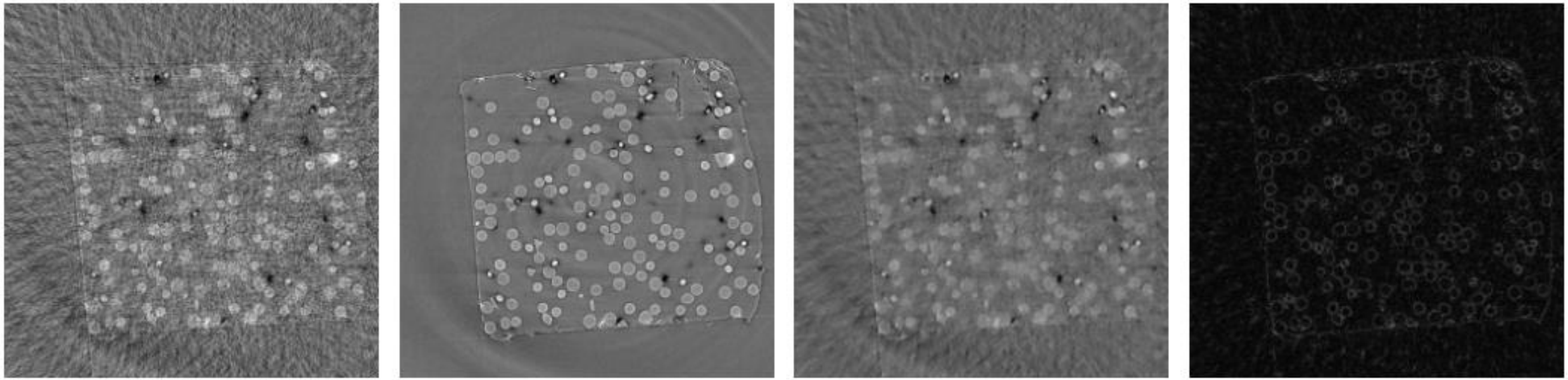
Agentic Exploration of Hyperparameters



Energy-Throughput



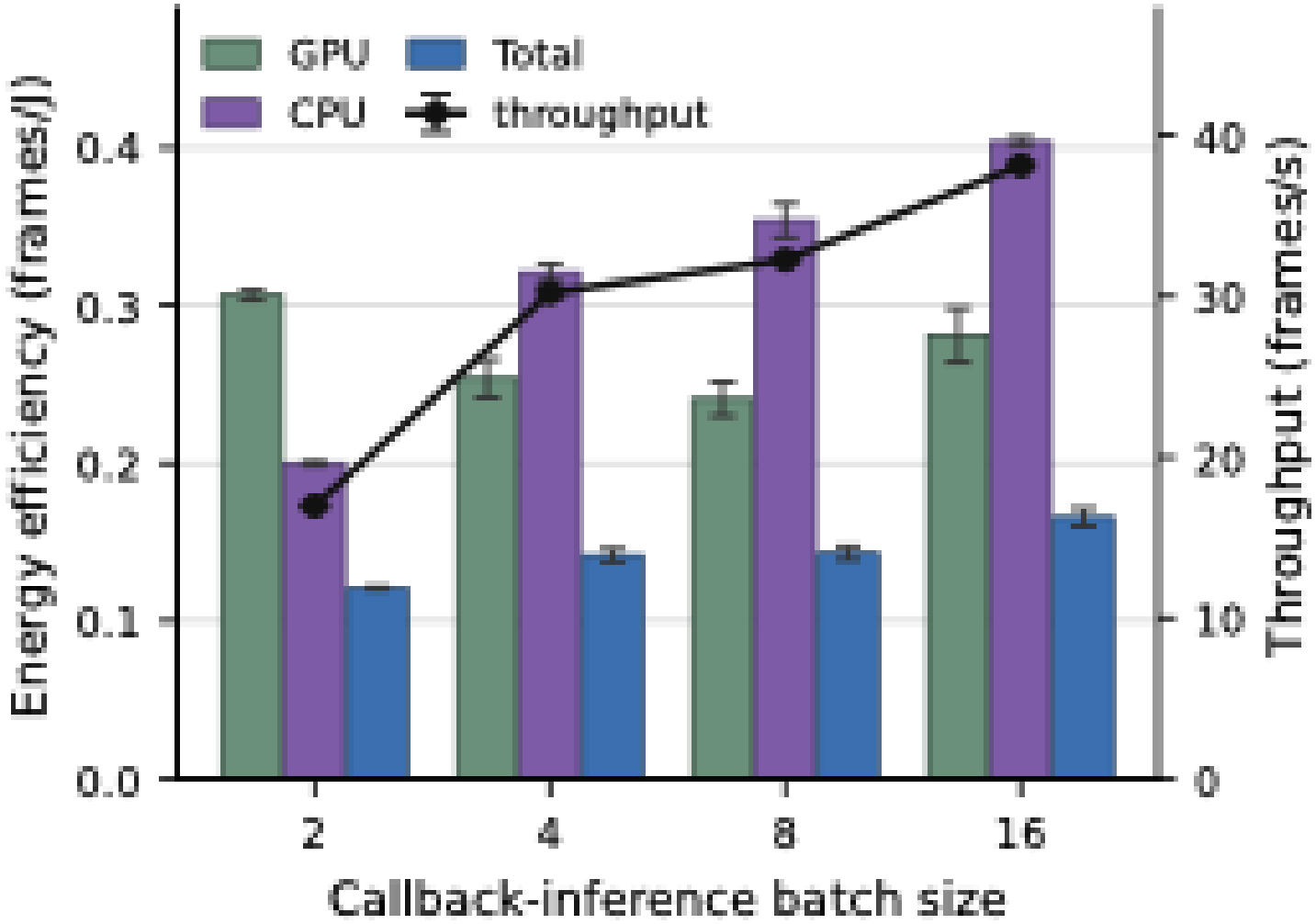
TomoGAN Pipeline Generated Artifacts



TomoGAN inference result from the Drava pipeline. From left to right: noisy input, ground truth, Drava/TomoGAN denoised output, and absolute error between the denoised output and ground truth

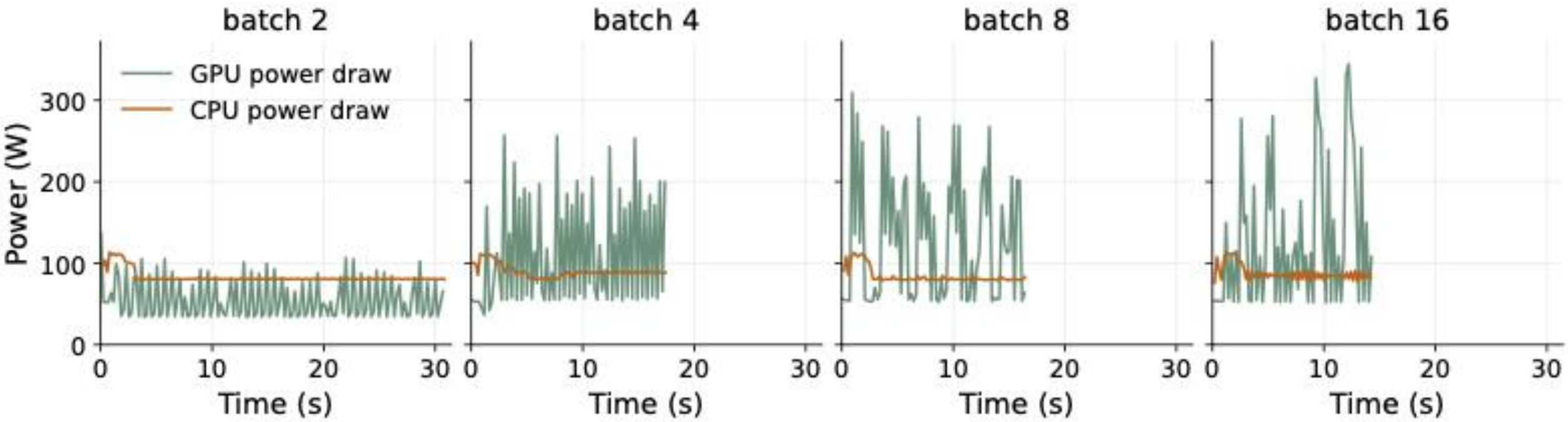
1. Liu, Z., Bicer, T., Kettimuthu, R., Gursoy, D., De Carlo, F., & Foster, I. (2020). TomoGAN: low-dose synchrotron x-ray tomography with generative adversarial networks: discussion. Journal of the Optical Society of America A, 37(3), 422-434.

Energy efficiency-throughput



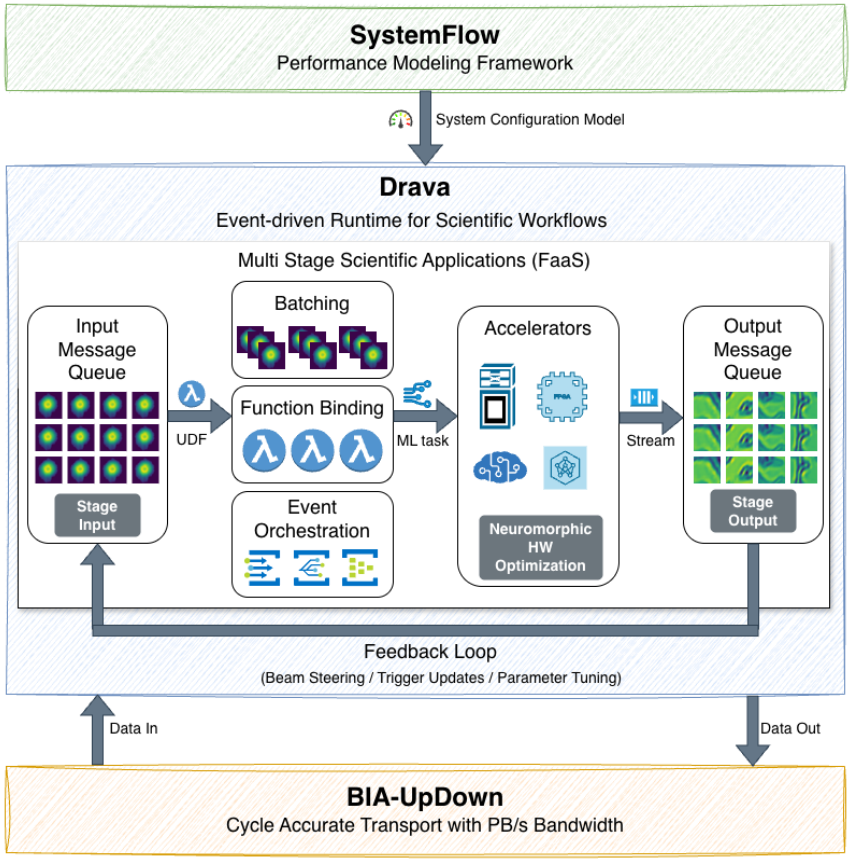
1. Liu, Z., Bicer, T., Kettimuthu, R., Gursoy, D., De Carlo, F., & Foster, I. (2020). TomoGAN: low-dose synchrotron x-ray tomography with generative adversarial networks: discussion. Journal of the Optical Society of America A, 37(3), 422-434.

Energy efficiency-throughput



1. Liu, Z., Bicer, T., Kettimuthu, R., Gursoy, D., De Carlo, F., & Foster, I. (2020). TomoGAN: low-dose synchrotron x-ray tomography with generative adversarial networks: discussion. Journal of the Optical Society of America A, 37(3), 422-434.

Future work

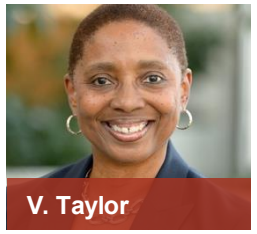


- Dynamic concurrency throttling based on energy consumption
- Integrate BIA-UpDown
- Get initial system configuration recommendation from SystemFlow 2.0
- Add feedback control mechanism
- Add more application examples (PtychoPINN, Pty-Chi, HEP)

Acknowledgement: Bia Team

Bia Co-Design for Energy-Efficiency

PI AND THRUST LEADS


V. Taylor


S. Habib


A. Chien

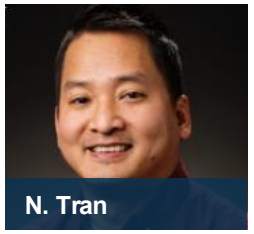

M. Hersam

Bia INDUSTRY ADVISORY BOARD

AMD · Lam Research · NVIDIA · Northrop Grumman

XRS AND HEP


S. Habib


N. Tran


T. Guruswamy


A. Paramonov


J. Hoff

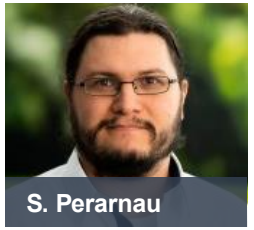

W. Olin-Ammentorp

SIM, ARCH, AND PM


A. Chien


A. Balaji


Y. Li


S. Perarnau

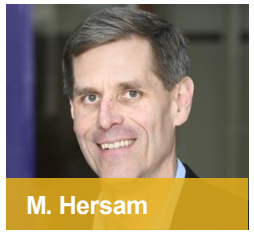

P. Darancet


W. Olin-Ammentorp


A. Shovon


X. Wu

MAT. AND DEVICES


M. Hersam


V. Sangwan

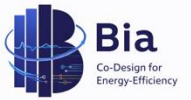

X. Ma


A. Bhattacharya


P. Darancet


Y. Li


W. Olin-Ammentorp





Thank you

Argonne NATIONAL LABORATORY